

Predicción de Insuficiencia Cardíaca mediante Machine Learning

Universidad de las Fuerzas Armadas ESPE
 Departamento de Ciencias de la Computación — Inteligencia Artificial
 Autores: Miguel Villacres · Miguel Sangucho | NRC: 30758
 Dataset: `heart.csv` (918 registros · 11 variables predictoras · 1 variable objetivo)

0. Instalación e importación de librerías

```
from google.colab import files
uploaded = files.upload()
```

Elegir archivos Ningún archivo seleccionado Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```

# Si estás en Google Colab, sube el archivo heart.csv con:
# from google.colab import files
# uploaded = files.upload() # <-- selecciona heart.csv

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, classification_report,
    confusion_matrix, ConfusionMatrixDisplay
)

import warnings
warnings.filterwarnings('ignore')

plt.style.use('seaborn-v0_8-whitegrid')
print('✅ Librerías cargadas correctamente')
```

✅ Librerías cargadas correctamente

1. Carga del Dataset

```

# -----
# Cambia la ruta si tu archivo está en otra ubicación
# En Colab: después de subir el archivo con files.upload(), usa 'heart.csv'
# En Jupyter local: usa la ruta completa o relativa
# -----

df = pd.read_csv('2heart.csv')

print(f'✅ Dataset cargado: {df.shape[0]} filas × {df.shape[1]} columnas')
df.head()
```

✅ Dataset cargado: 918 filas × 12 columnas

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

```
print('--- Tipos de datos y valores nulos ---')
df.info()
```

```
--- Tipos de datos y valores nulos ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                   918 non-null   int64
1   Sex                   918 non-null   object
2   ChestPainType         918 non-null   object
3   RestingBP             918 non-null   int64
4   Cholesterol           918 non-null   int64
5   FastingBS             918 non-null   int64
6   RestingECG            918 non-null   object
7   MaxHR                 918 non-null   int64
8   ExerciseAngina        918 non-null   object
9   Oldpeak               918 non-null   float64
10  ST_Slope              918 non-null   object
11  HeartDisease          918 non-null   int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

```
print('--- Estadísticas descriptivas ---')
df.describe().round(2)
```

```
--- Estadísticas descriptivas ---
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.00	918.00	918.00	918.00	918.00	918.00	918.00
mean	53.51	132.40	198.80	0.23	136.81	0.89	0.55
std	9.43	18.51	109.38	0.42	25.46	1.07	0.50
min	28.00	0.00	0.00	0.00	60.00	-2.60	0.00
25%	47.00	120.00	173.25	0.00	120.00	0.00	0.00
50%	54.00	130.00	223.00	0.00	138.00	0.60	1.00
75%	60.00	140.00	267.00	0.00	156.00	1.50	1.00
max	77.00	200.00	603.00	1.00	202.00	6.20	1.00

2. Análisis Exploratorio de Datos (EDA)

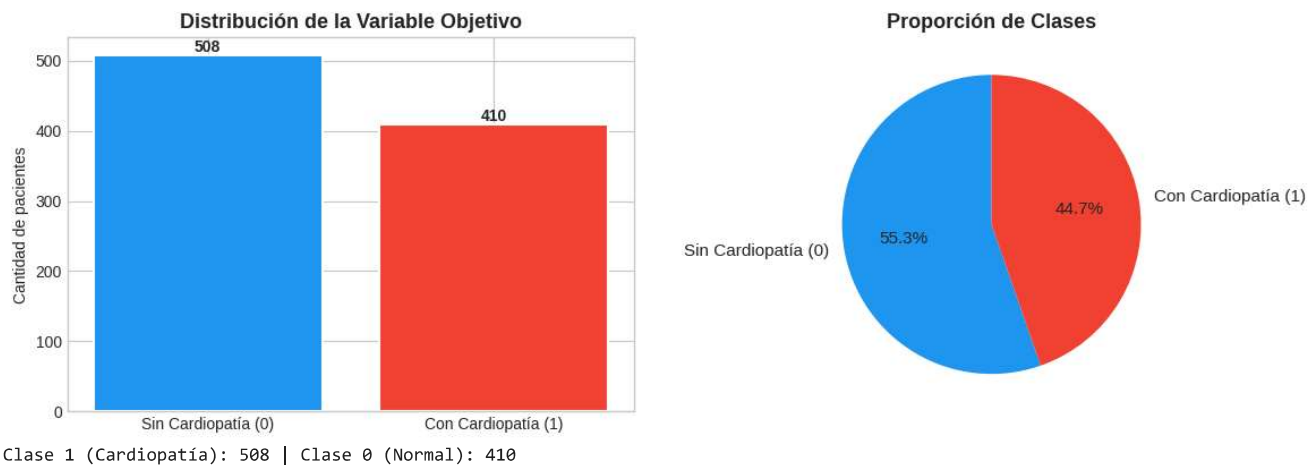
```
# Distribución de la variable objetivo
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

counts = df['HeartDisease'].value_counts()
labels = ['Sin Cardiopatía (0)', 'Con Cardiopatía (1)']
colors = ['#2196F3', '#F44336']

axes[0].bar(labels, counts.values, color=colors, edgecolor='white', linewidth=1.5)
axes[0].set_title('Distribución de la Variable Objetivo', fontsize=13, fontweight='bold')
axes[0].set_ylabel('Cantidad de pacientes')
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 5, str(v), ha='center', fontweight='bold')

axes[1].pie(counts.values, labels=labels, autopct='%1.1f%%',
            colors=colors, startangle=90, textprops={'fontsize': 11})
axes[1].set_title('Proporción de Clases', fontsize=13, fontweight='bold')

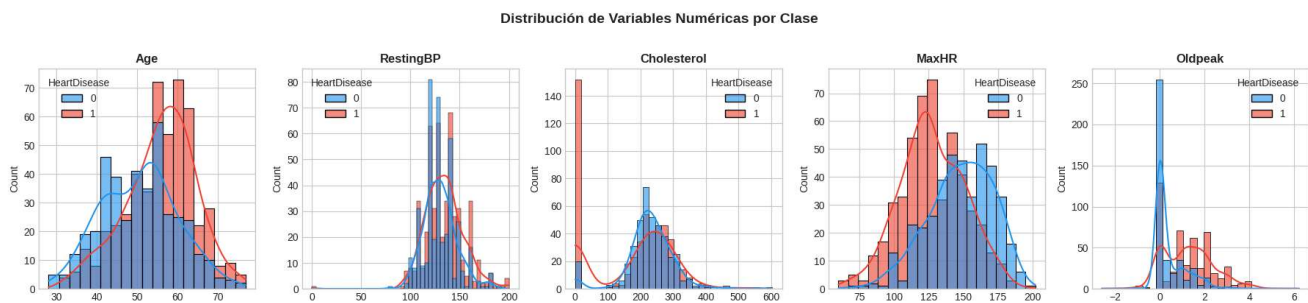
plt.tight_layout()
plt.show()
print(f'Clase 1 (Cardiopatía): {counts[1]} | Clase 0 (Normal): {counts[0]}')
```



```
# Distribución de variables numéricas
num_cols = ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak']

fig, axes = plt.subplots(1, len(num_cols), figsize=(18, 4))
for ax, col in zip(axes, num_cols):
    sns.histplot(data=df, x=col, hue='HeartDisease', kde=True,
                 palette={0: '#2196F3', 1: '#F44336'}, ax=ax, alpha=0.6)
    ax.set_title(col, fontweight='bold')
    ax.set_xlabel('')

plt.suptitle('Distribución de Variables Numéricas por Clase', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```

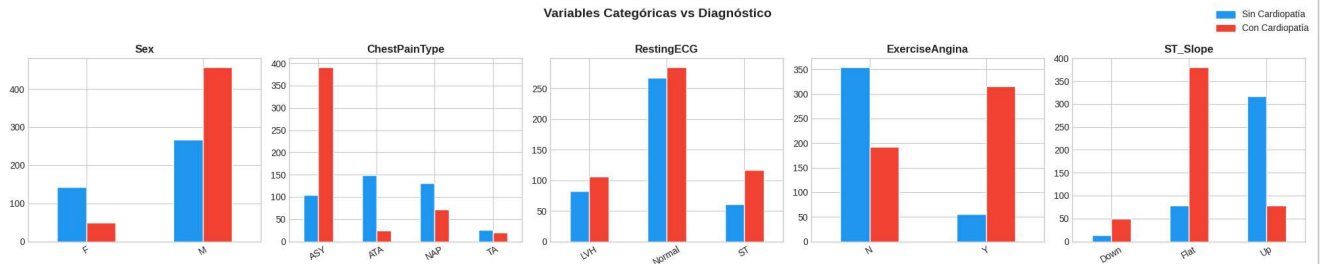


```
# Variables categóricas vs HeartDisease
cat_cols = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']

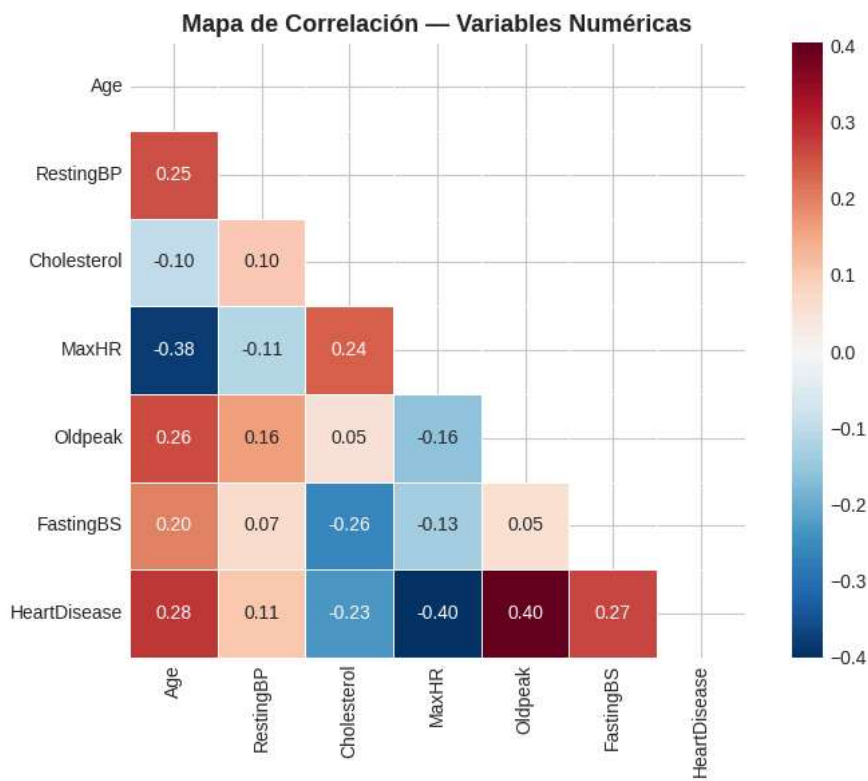
fig, axes = plt.subplots(1, len(cat_cols), figsize=(20, 4))
for ax, col in zip(axes, cat_cols):
    ct = pd.crosstab(df[col], df['HeartDisease'])
    ct.plot(kind='bar', ax=ax, color=['#2196F3', '#F44336'],
          edgecolor='white', legend=False)
    ax.set_title(col, fontweight='bold')
    ax.set_xlabel('')
    ax.tick_params(axis='x', rotation=30)

handles = [plt.Rectangle((0,0),1,1, color=c) for c in ['#2196F3', '#F44336']]
fig.legend(handles, ['Sin Cardiopatía', 'Con Cardiopatía'],
          loc='upper right', fontsize=10)

plt.suptitle('Variables Categóricas vs Diagnóstico', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()
```



```
# Mapa de correlación (solo numéricas)
plt.figure(figsize=(8, 6))
corr = df[num_cols + ['FastingBS', 'HeartDisease']].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='RdBu_r',
            center=0, linewidths=0.5, square=True)
plt.title('Mapa de Correlación — Variables Numéricas', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



3. Preprocesamiento de Datos

```
df_proc = df.copy()

# — Codificación de variables categóricas —
cat_cols = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']
le = LabelEncoder()
for col in cat_cols:
    df_proc[col] = le.fit_transform(df_proc[col])

print('Valores únicos tras codificación:')
for col in cat_cols:
    print(f' {col}: {sorted(df_proc[col].unique())}')
```

```
Valores únicos tras codificación:
Sex: [np.int64(0), np.int64(1)]
ChestPainType: [np.int64(0), np.int64(1), np.int64(2), np.int64(3)]
RestingECG: [np.int64(0), np.int64(1), np.int64(2)]
```

```
ExerciseAngina: [np.int64(0), np.int64(1)]
ST_Slope: [np.int64(0), np.int64(1), np.int64(2)]
```

```
# — División entrenamiento / prueba —
X = df_proc.drop('HeartDisease', axis=1)
y = df_proc['HeartDisease']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f'✅ Entrenamiento: {X_train.shape[0]} registros')
print(f'✅ Prueba: {X_test.shape[0]} registros')
```

```
✅ Entrenamiento: 734 registros
✅ Prueba: 184 registros
```

```
# — Normalización (para Regresión Logística) —
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print('✅ Normalización aplicada con StandardScaler')
```

```
✅ Normalización aplicada con StandardScaler
```

4. Entrenamiento de Modelos

```
# — Modelo 1: Random Forest —
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

print('✅ Random Forest entrenado')
```

```
✅ Random Forest entrenado
```

```
# — Modelo 2: Regresión Logística —
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_scaled, y_train)
y_pred_lr = lr_model.predict(X_test_scaled)

print('✅ Regresión Logística entrenada')
```

```
✅ Regresión Logística entrenada
```

5. Evaluación de Modelos

```
acc_rf = accuracy_score(y_test, y_pred_rf)
acc_lr = accuracy_score(y_test, y_pred_lr)

print(f'{'='*50}')
```

```
print(f' Random Forest → Accuracy: {acc_rf*100:.2f}%')
print(f' Regresión Logística → Accuracy: {acc_lr*100:.2f}%')
print(f'{'='*50}')
```

```
print('\n--- Reporte Random Forest ---')
print(classification_report(y_test, y_pred_rf,
    target_names=['Sin Cardiopatía', 'Con Cardiopatía']))

print('--- Reporte Regresión Logística ---')
print(classification_report(y_test, y_pred_lr,
    target_names=['Sin Cardiopatía', 'Con Cardiopatía']))
```

```
=====
Random Forest → Accuracy: 87.50%
Regresión Logística → Accuracy: 86.96%
=====

--- Reporte Random Forest ---
```

	precision	recall	f1-score	support
Sin Cardiopatía	0.87	0.84	0.86	82
Con Cardiopatía	0.88	0.90	0.89	102
accuracy			0.88	184
macro avg	0.87	0.87	0.87	184
weighted avg	0.87	0.88	0.87	184

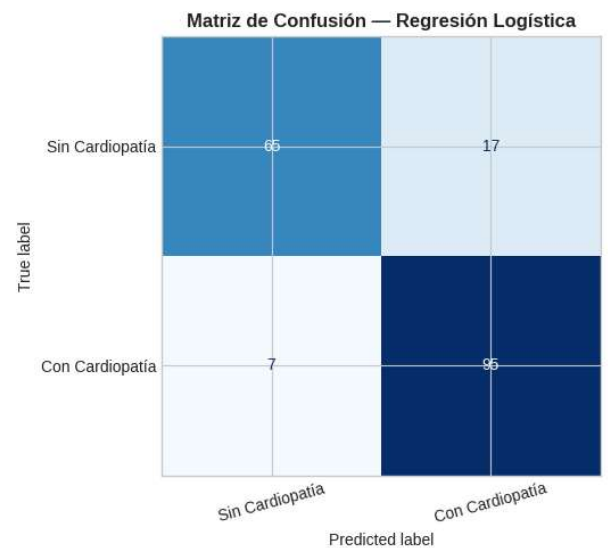
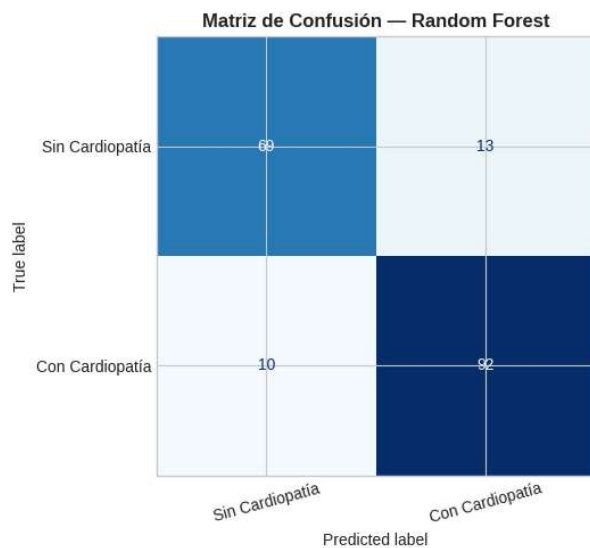
--- Reporte Regresión Logística ---

	precision	recall	f1-score	support
Sin Cardiopatía	0.90	0.79	0.84	82
Con Cardiopatía	0.85	0.93	0.89	102
accuracy			0.87	184
macro avg	0.88	0.86	0.87	184
weighted avg	0.87	0.87	0.87	184

```
# Matrices de confusión
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for ax, y_pred, title in zip(
    axes,
    [y_pred_rf, y_pred_lr],
    ['Random Forest', 'Regresión Logística']
):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(cm, display_labels=['Sin Cardiopatía', 'Con Cardiopatía'])
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(f'Matriz de Confusión — {title}', fontweight='bold', fontsize=12)
    ax.tick_params(axis='x', rotation=15)

plt.tight_layout()
plt.show()
```



```
# Comparativa de métricas
from sklearn.metrics import precision_score, recall_score, f1_score

metrics = {
    'Accuracy': [acc_rf, acc_lr],
    'Precision': [precision_score(y_test, y_pred_rf),
                  precision_score(y_test, y_pred_lr)],
    'Recall': [recall_score(y_test, y_pred_rf),
               recall_score(y_test, y_pred_lr)],
    'F1-Score': [f1_score(y_test, y_pred_rf),
                  f1_score(y_test, y_pred_lr)]
}

x = np.arange(len(metrics))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 5))
```

```

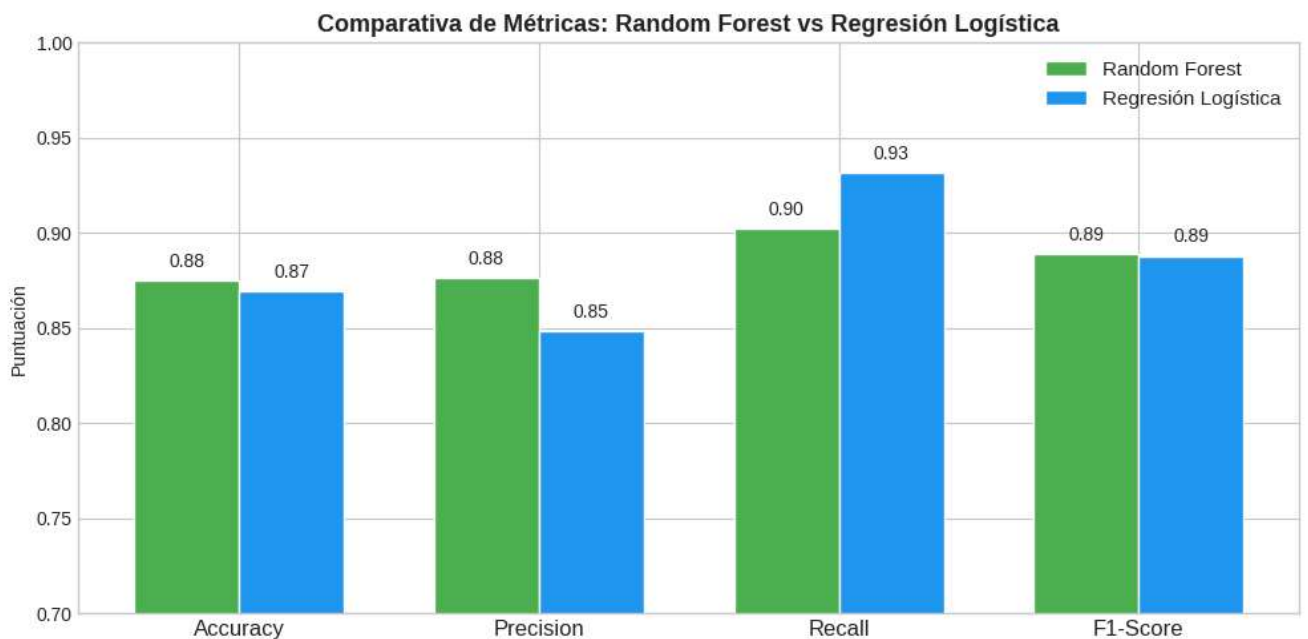
bars1 = ax.bar(x - width/2, [v[0] for v in metrics.values()],
              width, label='Random Forest', color='#4CAF50', edgecolor='white')
bars2 = ax.bar(x + width/2, [v[1] for v in metrics.values()],
              width, label='Regresión Logística', color='#2196F3', edgecolor='white')

ax.set_ylabel('Puntuación')
ax.set_title('Comparativa de Métricas: Random Forest vs Regresión Logística',
            fontsize=13, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(metrics.keys(), fontsize=12)
ax.set_ylim(0.7, 1.0)
ax.legend(fontsize=11)

for bar in bars1:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
           f'{bar.get_height():.2f}', ha='center', va='bottom', fontsize=10)
for bar in bars2:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
           f'{bar.get_height():.2f}', ha='center', va='bottom', fontsize=10)

plt.tight_layout()
plt.show()

```



6. Análisis de Importancia de Variables

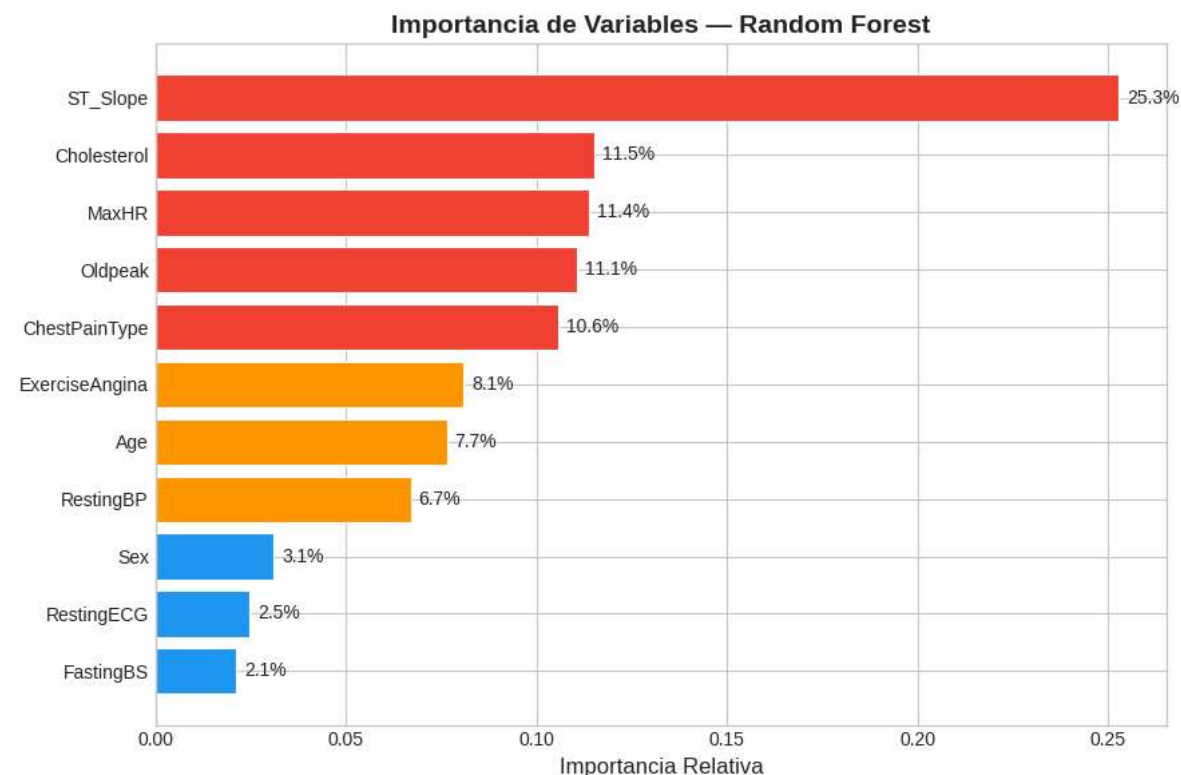
```

importances = rf_model.feature_importances_
feat_df = pd.DataFrame({
    'Variable': X.columns,
    'Importancia': importances
}).sort_values('Importancia', ascending=True)

plt.figure(figsize=(9, 6))
colors_bar = ['#F44336' if v > 0.10 else '#FF9800' if v > 0.06 else '#2196F3'
              for v in feat_df['Importancia']]
plt.barh(feat_df['Variable'], feat_df['Importancia'],
         color=colors_bar, edgecolor='white', linewidth=0.8)
plt.xlabel('Importancia Relativa', fontsize=12)
plt.title('Importancia de Variables - Random Forest', fontsize=14, fontweight='bold')
for i, (val, name) in enumerate(zip(feat_df['Importancia'], feat_df['Variable'])):
    plt.text(val + 0.002, i, f'{val*100:.1f}%', va='center', fontsize=10)
plt.tight_layout()
plt.show()

print(feat_df.sort_values('Importancia', ascending=False).to_string(index=False))

```



Variable	Importancia
ST_Slope	0.252837
Cholesterol	0.115252
MaxHR	0.113833
Oldpeak	0.110641
ChestPainType	0.105808
ExerciseAngina	0.080912
Age	0.076545
RestingBP	0.067069
Sex	0.031110
RestingECG	0.024690
FastingBS	0.021302

7. Validación Cruzada

```
cv_rf = cross_val_score(rf_model, X, y, cv=5, scoring='accuracy')
cv_lr = cross_val_score(lr_model, scaler.fit_transform(X), y, cv=5, scoring='accuracy')

print('Validación Cruzada 5-Fold:')
print(f' Random Forest → Media: {cv_rf.mean()*100:.2f}% ± {cv_rf.std()*100:.2f}%')
print(f' Regresión Logística → Media: {cv_lr.mean()*100:.2f}% ± {cv_lr.std()*100:.2f}%')
```

Validación Cruzada 5-Fold:

Random Forest → Media: 82.24% ± 3.92%

Regresión Logística → Media: 80.61% ± 2.34%

8. Predicción sobre Nuevos Pacientes

```
# Casos de prueba del informe
# Columnas: Age, Sex, ChestPainType, RestingBP, Cholesterol,
#           FastingBS, RestingECG, MaxHR, ExerciseAngina, Oldpeak, ST_Slope
#
# NOTA: los valores categóricos ya deben ser numéricos (LabelEncoder).
# En la codificación por defecto de LabelEncoder (alfabética):
# Sex: F=0, M=1
# ChestPainType: ASY=0, ATA=1, NAP=2, TA=3
# RestingECG: LVH=0, Normal=1, ST=2
# ExerciseAngina: N=0, Y=1
# ST_Slope: Down=0, Flat=1, Up=2
```

```
casos = pd.DataFrame([
    # Caso A: Hombre 63 años, dolor ASY, ST plana, Oldpeak=2.3
    [63, 1, 0, 130, 250, 0, 1, 110, 1, 2.3, 1],
    # Caso B: Mujer 48 años, dolor ATA, ST ascendente, MaxHR=172
    [48, 0, 1, 120, 200, 0, 1, 172, 0, 0.5, 2],
    # Caso C: Hombre 57 años, ExerciseAngina=Y, Cholesterol=289
    [57, 1, 0, 140, 289, 0, 1, 115, 1, 1.6, 1],
], columns=X.columns)

preds = rf_model.predict(casos)
probs = rf_model.predict_proba(casos)

for i, (pred, prob) in enumerate(zip(preds, probs)):
    label = '🔴 Con Cardiopatía' if pred == 1 else '🟢 Sin Cardiopatía'
    print(f'Caso {chr(65+i)}: {label} (probabilidad: {prob[pred]*100:.1f}%)')

Caso A: 🔴 Con Cardiopatía (probabilidad: 95.0%)
Caso B: 🟢 Sin Cardiopatía (probabilidad: 98.0%)
Caso C: 🔴 Con Cardiopatía (probabilidad: 97.0%)
```

Resumen del Pipeline

Etapa		Descripción
Carga	heart.csv	918 registros, 0 valores nulos
EDA		Distribución de clases, variables numéricas y categóricas, correlación
Preprocesamiento		LabelEncoder (5 vars cat.) + StandardScaler (para LR)
División		80% entrenamiento / 20% prueba (stratify)
Modelos		Random Forest (100 árboles) + Regresión Logística
Evaluación		Accuracy, Precision, Recall, F1-Score, Matrices de confusión
Importancia		ST_Slope > Oldpeak > Cholesterol > MaxHR > ExerciseAngina
Validación		Cross-validation 5-Fold